AdvancedPCBuilder

1. Problema ir idėja

Sistema vienu sakiniu: Multitenant SaaS platforma kompiuterių surinkėjams bei parduotuvėms, skirta automatiškai tikrinti komplektuojamų dalių fizinį bei elektrinį suderinamumą ir optimizuoti PC parinkimą.

Problema ir dabartinis procesas: Surinkėjai ir pardavėjai naudoja rankinius patikrinimus arba bazines e-parduotuvių filtrų sistemas. Trūkumas – neįvertinami sudėtingi fiziniai matmenys (pvz., CPU aušintuvo aukštis vs. RAM aukštis vs. korpuso plotis), PCIe linijų dalijimosi ribojimai ir momentiniai maitinimo blokų pikai.

Nauda: Sumažėja surinkimo klaidų bei grąžinimų skaičius, sutaupomas naudotojų laikas ir automatizuojamas reikalavimus atitinkančių stalo kompiuterių sąmatų sudarymas.

Naudotojai: Kompiuterinės technikos parduotuvės bei individualūs vartotojai, valdantys savo inventorių ir klientų komplektacijas.

Prielaidos: Daroma prielaida, kad parduotuvėms ir surinkėjams reikalingas tikslus fizinių matmenų bei elektrinių parametrų patikrinimo sistema.

2. Apimtis

Funkcija	Ką naudotojas galės atlikti	Pagrindinis modulis ar pagalbinė funkcija
Registracija ir nustatymai	Parduotuvė / surinkėjas susikuria paskyrą, valdydamas savo krepšelį ir antkainius.	Pagalbinė funkcija
Komplektacijos surinkimas	Naudotojas pasirenka komponentus iš sąrašo ir formuoja PC komplektaciją.	Pagalbinė funkcija
Suderinamumo ir ribojimų tikrinimas	Variklis atlieka fizinių matmenų atitikimo, elektros sunaudojimo ir PCIe linijų ribojimų skaičiavimus.	Pagrindinis modulis
AI komplektacijos generatorius	Naudotojas įveda biudžetą ir paskirtį, o dirbtinis intelektas pateikia pradinę komplektaciją patikrinimui.	Pagalbinė funkcija (AI)

Į kursinio darbo apimtį neįeina: Realių mokėjimų apdorojimas, fizinis prekių pristatymo sekimas.

3. Pagrindinis modulis

Pavadinimas ir atsakomybė: Komponentų fizinio ir elektrinio suderinamumo variklis. Jis atsakingas už sudėtingų daugiakriterių taisyklių (matmenų, maitinimo, jungčių konfliktų) patikrinimą ir validavimą, kai surenkamas kompiuteris.

Logika, kurią reikės projektuoti ir testuoti: Skaičiavimai apims trimatės erdvės apribojimus (pvz., CPU aušintuvo aukščio, RAM modulių aukščio ir korpuso pločio kryžminis konfliktas), sistemos bendro šiluminio galingumo (TDP) ir momentinių apkrovų (transient spikes) modeliavimą, lyginant su maitinimo bloko (PSU) galimybėmis, bei magistralių (PCIe) pralaidumo apribojimus (pvz., ar įstačius tam tikrą M.2 diską neatsijungia SATA portai).

Įvestis: Vartotojo suformuotas pasirinktų PC komponentų sąrašas JSON formatu. Pavyzdžiui: pasirenkamas „AMD Ryzen 7 7800X3D“ procesorius, „Noctua NH-D15“ aušintuvas, aukšto profilio „Corsair Dominator“ RAM moduliai, „NZXT H510“ korpusas ir „Corsair 750W“ maitinimo blokas.

Išvestis: Suderinamumo ataskaita JSON formatu su bendru statusu ir konkrečiais klaidų arba įspėjimų pranešimais. Pavyzdžiui: Statusas: Atmesta, su pranešimais „Aušintuvo ir RAM aukščių suma viršija korpuso palaikomą plotį 4 mm“ ir „Maitinimo bloko galia ribinė šiai komplektacijai“.

Veikimo eiga:

1. Sistema priima pasirinktų komponentų užklausą.
2. Iš duomenų bazės (arba cache atminties) ištraukiami detaliųjų specifikacijų duomenys.
3. Paleidžiamas fizinio suderinamumo algoritmas (tikrinami milimetrai, lizdai, form faktoriai).
4. Paleidžiamas elektrinio ir magistralių suderinamumo algoritmas (TDP sumos, PCIe linijos).
5. Sugeneruojamas atsakymas su klaidomis (jei konfliktas) arba patvirtinimu.
6. Atsakymas grąžinamas vartotojo sąsajai.

Taisyklės arba sprendimo žingsniai

1. CPU aušintuvo aukštis (mm) privalo būti mažesnis arba lygus korpuso nurodytam maksimaliam palaikomam aušintuvo aukščiui.
2. Visų pasirinktų komponentų bendra suminė galia (TDP) pridėjus 20 % atsargos koeficientą negali viršyti maitinimo bloko (PSU) nominalios galios.
3. Vaizdo plokštės ilgis (mm) turi būti mažesnis nei korpuso maksimalus GPU ilgis, iš kurio papildomai atimamas priekyje sumontuoto aušinimo radiatoriaus storis (jei toks pasirinktas).

Scenarijai būsimiems testams

Scenarijus	Pradinės sąlygos ir konkreti įvestis	Veiksmas	Tikslus laukiamas rezultatas
Įprastas atvejis	Standartinė komplektacija: 65W TDP procesorius, mATX motininė plokštė, vidutinio dydžio korpusas, standartinis RAM, 600W PSU.	Vartotojas (arba testas) inicializuoja suderinamumo patikrinimą.	Grąžinamas statusas „Patvirtinta“ ir tuščias klaidų/įspėjimų sąrašas.
Ribinis atvejis arba konfliktas	Sistemos suminė galia su 20 % atsarga lygiai atitinka pasirinkto PSU maksimalią galią (pvz., reikalauja 750W ir įdėtas 750W PSU).	Inicializuojamas suderinamumo patikrinimas.	Grąžinamas statusas „Patvirtinta“, bet pridedamas „Įspėjimas“, nurodantis, kad maitinimo galia yra ribinė atnaujinimams ateityje.
Klaida arba neįmanomas rezultatas	Pasirinktas „E-ATX“ formato mot. plokštė, bet pasirinktas kompaktiškas „Mini-ITX“ formato korpusas.	Inicializuojamas suderinamumo patikrinimas.	Grąžinamas statusas „Atmesta“ su aiškiu klaidos pranešimu: „Motininės plokštės formatas E-ATX netelpa į pasirinktą Mini-ITX korpusą“.

Jei modulis naudoja AI: Netaikoma.

4. Kokybės atributas

Pasirinktas atributas: Spartumas (Našumas / Performance).

Kodėl svarbus šiai sistemai: Komplektuojant kompiuterį e-parduotuvėse, vartotojai tikisi momentinio (realiu laiku) suderinamumo patikrinimo po kiekvieno detalės pakeitimo. Kadangi tai yra SaaS sistema, ji vienu metu aptarnaus daugybės skirtingų klientų (tenantų) vartotojus. Sudėtingas matematinių ir loginių taisyklių tikrinimo variklis privalo veikti greitai ir neblokuoti serverio resursų, antraip sistema taps nepatogi naudoti.

Tikrinimo scenarijus ir sąlygos: Bus simuliuojama didelė lygiagreti API apkrova. Į suderinamumo tikrinimo modulį (API) vienu metu bus išsiųsta 500 užklausų, kur kiekviena užklausa turės sudėtingą, 8–10 skirtingų komponentų komplektaciją iš atsitiktinių tenantų.

Sėkmės kriterijus: 95 % API patikrinimo užklausų (p95) apdorojamos ir atsakymas grąžinamas greičiau nei per 200 milisekundžių. Neįvykusių užklausų (klaidų, pvz., HTTP 500 ar *timeout*) procentas lygus 0 %.

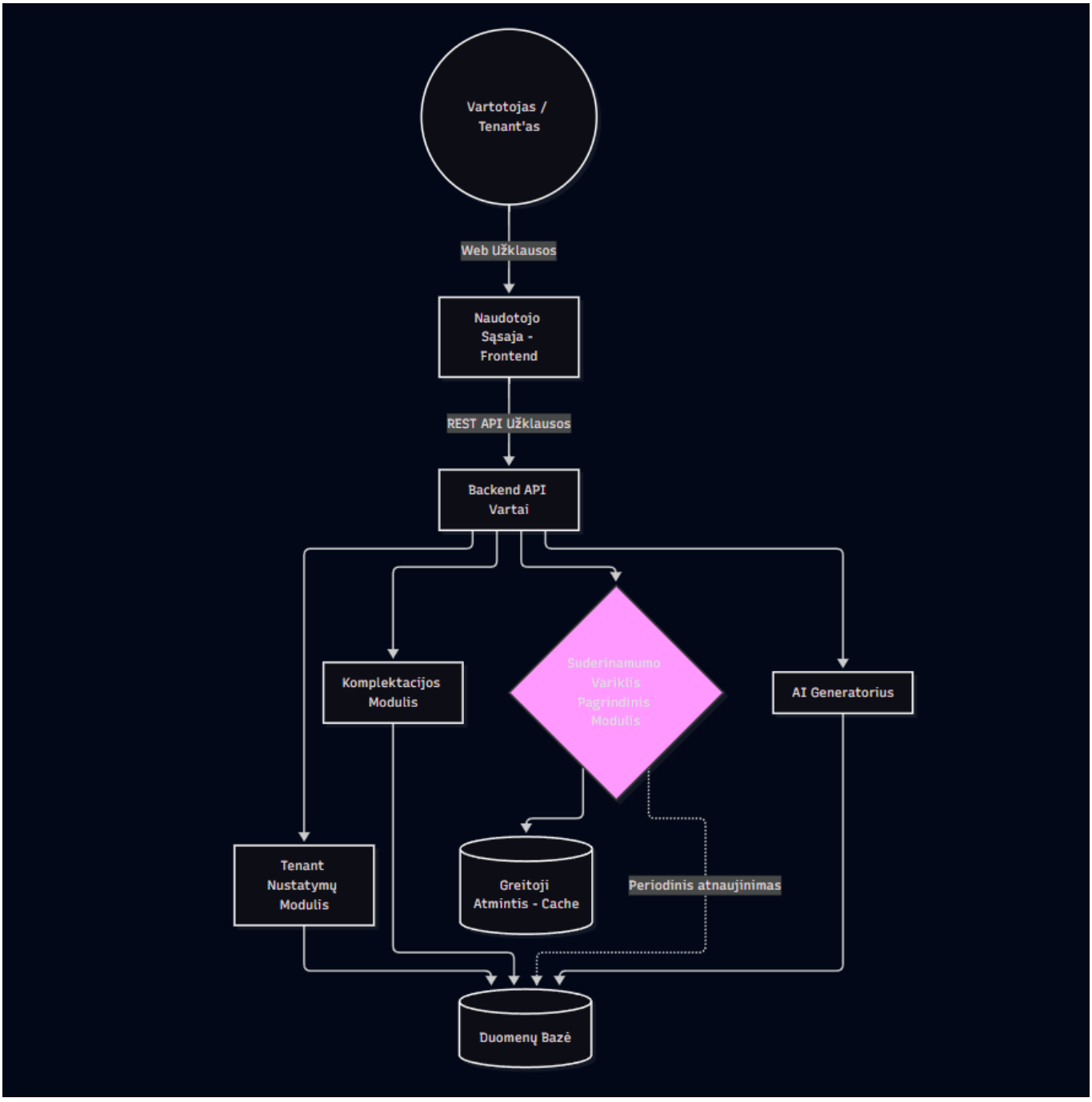
Numatytas projektavimo sprendimas: Sistema bus projektuojama naudojant *stateless* (būsenos nesaugančią) architektūrą. Siekiant išvengti lėtų kreipinių į duomenų bazę skaičiavimų metu, visos komponentų specifikacijos ir taisyklės bus užkraunamos į greitąją operatyviąją atmintį.

Kaip patikrinsiu vėlesniame etape: Sukursiu ir paleisiu automatizuotą apkrovos testavimo skriptą, naudodamas specializuotą įrankį (pvz., „k6“, „JMeter“ arba „NBomber“). Įrankis sugeneruos detalią ataskaitą, įrodančią API atsako laiko matavimus prie nurodytos apkrovos.

Sprendimo kaina arba ribojimas: Sprendimas naudoti greitąją atmintį (*caching*) padidina serverio operatyviosios atminties (RAM) sąnaudas.

5. Pradinė sistemos struktūra

Paprasta schema



Sistemos dalis	Atsakomybė
Naudotojo sąsaja (Frontend)	Priima vartotojo įvestį, atvaizduoja vizualų komponentų pasirinkimą ir klaidas/įspėjimus, grįžtančius iš API.
Backend API	Priima HTTP užklausas iš Frontend'o, tikrina autentifikaciją, nustato iš kokio Tenant'o ateina užklausa ir nukreipia ją atitinkamam moduliui.
Suderinamumo patikrinimas (Pagrindinis modulis)	Vykdo skaičiavimus fiziniams (matmenų) ir elektriniams suderinamumams tikrinti pagal nustatytas taisykles naudojant duomenis iš greitosios atminties.

Sistemos dalis	Atsakomybė
Duomenų bazė	Saugo komponentų specifikacijas, tenant'ų nustatymus, vartotojų anketas ir išsaugotas komplektacijas.
Greitoji atmintis (Cache)	Laiko dažniausiai naudojamų komponentų matmenų/galios taisykles, kad suderinamumo variklis galėtų atlikti skaičiavimus išvengiant lėtų DB užklausų.

Planuojamos technologijos ir pasirinkimo priežastys:

- **C# (.NET 8 / ASP.NET Core)** – Backend API ir pagrindinės skaičiavimo logikos kūrimui. Pasirinkta dėl didelio našumo, stipraus tipizavimo (strongly-typed) bei asmeninio susidomėjimo šia technologija ir noro tobulinti žinias objektinio programavimo srityje.
- **TypeScript ir React**– Vartotojo sąsajai kurti. TypeScript pasirinktas dėl kodo patikimumo užtikrinimo (tipų saugumo), o React leidžia lengvai kurti interaktyvias sąsajas.
- **PostgreSQL** – Pagrindinėms sistemos reliacinėms duomenims (tenant'ams, komponentų katalogui) saugoti. Pasirinkta dėl patikimumo, atviro kodo ir lengvo integravimo su .NET per Entity Framework.
- **Redis** – Greitajai (in-memory) atminčiai realizuoti, kurios reikės pagrindinio modulio našumui (kokybės atributui) pasiekti.
- **Docker** – Sistemos konteinerizavimui, užtikrinant „SaaS / Cloud-first“ principą, leidžiantį sistemą paleisti viena komanda iš README failo.
- **GitHub** – Kodo versijavimui ir projekto dokumentacijai, kaip to reikalauja dėstytojas.

6. AI panaudojimas

AI rengiant šį dokumentą

Priemonė ir užduotis	Ką panaudojau	Ką atmečiau arba perrašiau ir kodėl	Kaip patikrinau
Pagrindines sistemos problemos bei naudos apibrėžimas	Gemini	Ištriniau nereikalingus sakynius, daug nereikalingų paaiškinimų skliaustuose	Perskaičiau
Sugeneruoti pradinę sistemos struktūros schemą (Mermaid formatu)	Gemini	Palikau kaip yra	Peržiurėjau
Technologijų pasirinkimas	Gemini	Pašalinau Angular ir Vue, nes nesu su jais dirbęs, pakeičiau pagrindinę programavimo kalba į C#, nes noriu su ją bandyti įgyvendinti projektą.	Perskaičiau
Tolesnių darbo plano lentelės užpildymas	Gemini	Atmečiau planus, kurie buvo netinkami	Perskaičiau sugeneruota lentelę, išanalizavau kiekvieną įrašą

Planuojamas AI naudojimas kuriant sistemą

Į IDE integruoti įrankiai, autofill, kalbos modeliai (Copilot, Gemini)

Kur ir kam naudosiu AI: Rašant pasikartojantį kodą, generuojant netikrus PC komponentų duomenis duomenų bazei, sprendžiant „Docker“ konfigūracijos klaidas bei ieškant patarimų, kaip geriau realizuoti C# skaičiavimo taisykles.

Kaip tikrinsiu pasiūlymus ir sugeneruotą kodą: Kiekvieną sugeneruotą kodo bloką perskaitysiu ir analizuosiu. Pagrindinio modulio logikai pritaikysiu Unit testus. Nenaudosiu kodo, kurio pats negalėčiau paaiškinti atsiskaitymo metu.

Ar AI bus sistemos funkcionalumo dalis: Taip. AI bus naudojamas kaip papildoma funkcija. Jo paskirtis – pagal vartotojo įvestą tekstą (pvz., biudžetą ir paskirtį) sugeneruoti pradinį kompiuterio komponentų sąrašą, kurį vėliau validuos pagrindinis suderinamumo patikrinimo įrankis.

7. Tolesnių darbų planas

Darbas	Apčiuopiamas rezultatas	Planuojama darbų seka
Projekto ir Docker aplinkos paruošimas	Sukurta repozitorija, veikiantis tuščias .NET API ir PostgreSQL paleidžiami viena „docker compose up“ komanda.	1
Duomenų bazės struktūra ir pavyzdiniai duomenys	Sukurtos lentelės komponentams ir tenant'ams. Duomenų bazė užpildyta keliais testiniais procesoriais, aušintuvais, korpusais ir maitinimo blokais su jų matmenimis ir galia.	2
Pagrindinio modulio logika	Sukurtos C# klasės ir taisyklės, kurios paima komponentų sąrašą ir patikrina fizinius bei elektrinius limitus (be API prijungimo). Parašyti Unit testai.	3
API Endpoints bei Redis integracija	Sukurti REST API *endpointai*, priimantys užklausas. Variklis sujungtas su *cache*, užtikrinant greitą veikimą.	4
Vartotojo sąsajos sukūrimas	Paprastas internetinis puslapis, kuriame galima išskleidžiamųjų sąrašų pagalba pasirinkti komponentus ir iškviesti API patikrinimą.	5
AI komponento integracija	Pridėtas tekstinis įvesties laukas, kur API per kalbos modelį sugeneruoja pradinę kompiuterio komplektaciją pagal vartotojo biudžetą.	6

Būsimo prototipo veikimo scenarijus:

Prototipo demonstravimo metu bus parodyta, kaip vartotojas išsirenka komponentus imituotoje e-parduotuvės aplinkoje. Pirmiausia bus pateikta nesuderinama komplektacija: galingas procesorius, milžiniškas CPU aušintuvas, bet per siauras korpusas ir per silpnas maitinimo blokas. Paspaudus „Tikrinti“, sistema turės per mažiau nei 200 ms (demonstruojant našumą) grąžinti atmetimo atsakymą su tiksliomis klaidomis (pvz., trūksta 5 mm aušintuvui; PSU trūksta 150W galios). Antru žingsniu tie du komponentai bus pakeisti į tinkamus, sistema vėl greitai perskaičiuos ir grąžins patvirtinimą, kad kompiuterį galima sėkmingai surinkti.

Rizika arba neaiškumas	Kaip patikrinsiu arba sumažinsiu
Suderinamumo taisyklių variklis pasidarys per daug sudėtingas ir bus sunku rasti klaidą.	Taisykles kursiu izoliuotai. Kiekvienai taisyklei parašysiu atskirus *Unit testus* prieš pereinant prie sekančios taisyklės.
Redis integravimas neveiks sklandžiai arba grąžins senus duomenis.	Sistemą iš pradžių suprojektuosiu taip, kad ji veiktų tiesiogiai su duomenų baze. Redis sluoksnį pridėsiu tik tada, kai pagrindinė logika bus pilnai veikianti, kaip optimizacijos priedą.

Šaltiniai, jei naudojote

Nenaudojau.